2025/2026	AP – PROJET RESTO
BTS SIO 1SIOB	Auteur : Groupe 5
	Date de rédaction : 29/09/2025

Itération 2 : résolution de tickets

Compte Rendu de la Réunion de la séance 3 : 1

Ticket 3 : 1

Ticket 4 : 2

Ticket 5 : 3

Ticket 6 : 4

Ticket 7 : 5

Ticket 8 : 6

Ticket 9 : 6

Ticket 10 : faille de sécurité sur la page d'authentification 7

Ticket 11 : 8

Compte Rendu de la Réunion de la séance 3 :

Lors de cette réunion nous avons fini de mettre en place les différents tickets de l'itération 2 puis nous nous sommes partagé le travail avec 3 tickets chacun :

Titouan : Tickets 8, 9 et 11

Nikauly : Tickets 4, 5 et 7

Valentin : Tickets 3, 6 et 10

Une fois cela fait, nous avons commencé le travail en réalisant les différentes tâches tout en écrivant les comptes rendus ci-dessous.

Ticket 3 :

Contexte :

On souhaite que le numéro de l'équipe apparaisse sur la page d'accueil sous la forme du texte suivant : mise à jour par équipe X dans le pied de page.

Solution :

Pour cela, on ajoute simplement une nouvelle ligne à la fin de la vue de l'accueil permettant d'afficher la phrase souhaitée.

```
Classement basé sur les critiques de nos utilisateurs.
<br>
<p style="text-align: right">Mise à jour réalisée par l'équipe numéro 5.</p>
```

Une fois cela fait, on remarque que celle-ci se retrouve bien en pied de page sur la page d'accueil comme souhaité.

Classement basé sur les critiques de nos utilisateurs.

Mise à jour réalisée par l'équipe numéro 5.

Ticket 4 :

On souhaite gerer les requettes inexistant ainsi que afficher un message sur la page d'accueilpour prevenir le client que le blob n'existe pas

Solution :

Etape 1 : on gère les actions inexistant

Source History

```php
function controleurPrincipal(string $action) : string {
    $lesActions = array();
    $lesActions["defaut"] = "accueil.php";
    $lesActions["cgu"] = "cgu.php";
    $lesActions["liste"] = "listeRestos.php";
    $lesActions["detail"] = "detailResto.php";
    $lesActions["recherche"] = "rechercheResto.php";
    $lesActions["connexion"] = "connexion.php";
    $lesActions["deconnexion"] = "deconnexion.php";
    $lesActions["profil"] = "monProfil.php";
    $lesActions["updProfil"] = "updProfil.php";
    $lesActions["inscription"] = "inscription.php";
    $lesActions["aimer"] = "aimer.php";
    $lesActions["noter"] = "noter.php";
    $lesActions["commenter"] = "commenter.php";
    $lesActions["supprimerCritique"] = "supprimerCritique.php";


    if (array_key_exists ( $action , $lesActions )){
        return $lesActions[$action];
    }
    else{
        $_SESSION['messageErreur'] = "la page demande n'existe pas";
        return $lesActions["defaut"];
    }

}
```

Etape 2 : on affiche message pour prevenir le client

```php
?>
<?php
//affichage d'erreur si blob n'existe pas
if (isset($_SESSION['messageErreur'])) {
    echo "<div class='alerte-erreur'>" . $_SESSION['messageErreur'] . "</div>";
    unset($_SESSION['messageErreur']);
}
?>
```

Ticket 5 :

Contexte :

On suite amélioré la sécurité des mots de passe, le programme utilise la methode crypt() et on suite maintenant utiliser la méthode password_hash() qui s'avère être plus sécurisée que la méthode crypt()

Solution :

Dans un premier temps nous devant gère l'enregistrement des mots de passes pour cela nous aller modifier la fonction updateMdp() :

On enlevé la méthode crypt() a la ligne 141 et on la change pour la méthode password_hash:

```php
public static function updateMdp(int $idU, string $mdpClair): bool {
    $ok = false;
    try {
    $requete = "UPDATE utilisateur SET mdpU = :mdpU WHERE idU = :idU";
    $stmt = Bdd::getConnexion()->prepare($requete);
    $mdpHash = password_hash($mdpClair, PASSWORD_DEFAULT);
    $stmt->bindValue(':mdpU', $mdpHash, PDO::PARAM_STR);
    $stmt->bindValue(':idU', $idU, PDO::PARAM_INT);
    $ok = $stmt->execute();
    } catch (PDOException $e) {
        throw new Exception("Erreur dans la méthode " . get_called_class() . "::updateMdp : <br/>" . $e->getMessage());
    }
    return $ok;
}
```

Donc si on crée un nouvel utilisateur on pourra remarquer que le mot de passe s'est bien enregistré avec la mon cryptage comme dans l'image en desous on peut voir que le l'utilisateur "testpasswordhash" et l'ulisateur "yann@lechambon.fr " n'on pas la même methode de cryptage au niveau du mots de passe :

Options supplémentaires

			idU	mailU	mdpU	pseudoU
Éditer	Copier	Supprimer	1	alex.garat@gmail.com	1zvN5hYSQSQDFUIQSdufUQSDFznHF5osT.	@lex
Éditer	Copier	Supprimer	2	jj.soueix@gmail.com	1zvN5hYMI$SDFGSDFGJqJSDJF.	drskott
Éditer	Copier	Supprimer	3	mathieu.capliez@gmail.com	seSzpoUAQgII.	pich
Éditer	Copier	Supprimer	4	michel.garay@gmail.com	1zvN5hYMI$VSatLQ6SDFGdsfgznHF5osT.	Mitch
Éditer	Copier	Supprimer	5	nicolas.harispe@gmail.com	1zvNDSFQSdfqsDfQsdfsT.	Nico40
Éditer	Copier	Supprimer	6	test@bts.sio	seSzpoUAQgII.	testeur SIO
Éditer	Copier	Supprimer	7	yann@lechambon.fr	sej6dETYI/ea.	yann
Éditer	Copier	Supprimer	8	testpassword_hash@bts.sio	$2y$10$ZbWNQoy0JFHA.hCz1xew.uFdqKaUFj.caDGvsRF6Ldk	sio

J'ai ensuite aussi changer la methode login pour que l'on puisse ce connecter en utilisant la methode password_verify(), cette methode prend en paramètre deux variables ici la le mdpU qui est le mots de passe que l'utilisateur a rentré pour ce connecter mais aussi le mdpBd qui est le mots de passe qui est crypté dans la base de données , elle les compare est nous renvoie un booléen.

```php
function login(string $mailU, string $mdpU): void {
    if (!isset($_SESSION)) {
        session_start();
    }
    // Rechercher les données relatives à cet utilisateur
    $util = UtilisateurDAO::getOneByMail($mailU);
    // Si l'utilisateur est connu (mail trouvé dans la BDD)
    if (!is_null($util)) {
        $mdpBD = $util->getMdpU();
        $idU = $util->getIdU();

        // Si le mot de passe saisi correspond au mot de passe "haché" de la BDD
        if(password_verify($mdpU, $mdpBD) == true){
            $_SESSION["idU"] = $idU;        // la clef est idU désormais
            $_SESSION["mailU"] = $mailU;
            $_SESSION["mdpU"] = $mdpBD;
        }elseif(trim($mdpBD) == trim(crypt($mdpU, $mdpBD))) {
            // le mot de passe est celui de l'utilisateur dans la base de donnees
            $_SESSION["idU"] = $idU;        // la clef est idU désormais
            $_SESSION["mailU"] = $mailU;
            $_SESSION["mdpU"] = $mdpBD;
        }
    }
}
```

J'ai aussi ajouté un elseif pour pouvoir gérer les mots de passe qui ont déjà été enregistré dans la base de données avec la méthode crypt().

Ticket 6 :

Contexte :

Après consultation des données enregistrées dans la base de données, on constate que le classement des des quatre restaurants obtenant les meilleures critiques est faux :

Repérage du problème :

Ici, c'est un problème de requête SQL dans notre RestoDAO et principalement dans la fonction "getTop4" qui réalise une somme avec la fonction "SUM" et donc qui va mettre dans le top 4 les restaurants avec la note la plus élevée mais en additionnant toutes les notes.

```
try {
    $requete = "SELECT SUM(note) AS NotesCumulees, r.
                FROM resto r
                INNER JOIN critiquer c ON r.idR = c.id
```

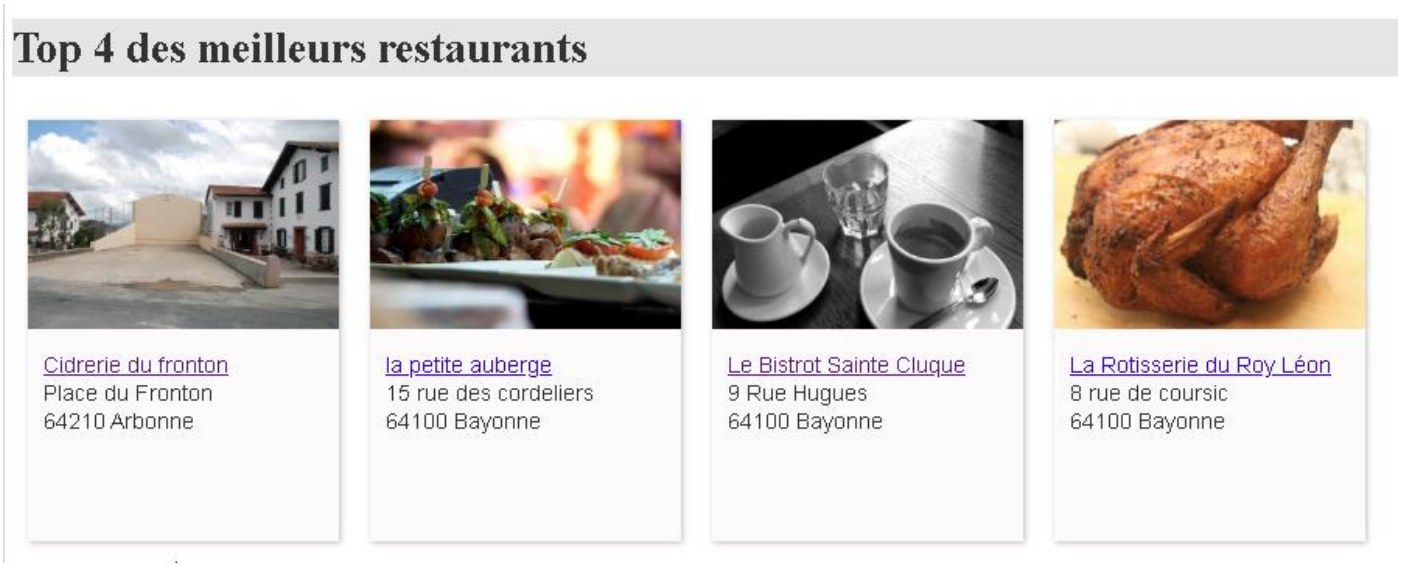
Ainsi un resto avec 10 notes à une étoile sera mieux placé qu'un restaurant avec une note à 5 étoiles.

Solution :

Pour régler ce problème, on transforme le "SUM" en "AVG" qui va lui cette fois nous donner la moyenne et non la somme ce qui nous donnera bien les restaurants les mieux notés pour notre top 4.

```
public static function getTop4(): array {
    $lesObjets = array();
    try {
        $requete = "SELECT AVG(note) AS NotesCumulees, r.i
                    FROM resto r
```

Si on vérifie, le nouveau top 4 affiché ensuite est bien en correspondance avec les meilleurs résultats dans la base de données si on y entre la requête sans le "LIMIT 4".



Ticket 7 :

On souhaite afficher la note moyenne d'un restaurant non arrondi :

Solution :

```php
$lesPhotos = $unResto->getLesPhotos();
$critiques = $unResto->getLesCritiques();
$noteMoy = CritiqueDAO::getNoteMoyenneByIdR($idR);
$idU = getIdULoggedOn();
$mailU = getMailULoggedOn();
$aimer = AimerDAO::estAimeById($idU, $idR);
$maCritique = CritiqueDAO::getOneById($idR, $idU);
$monCommentaire = "";
if ($maCritique) {
    $monCommentaire = $maCritique->getCommentaire();
}
$titre = "detail d'un restaurant";
```

```php
<span id="note">
    <?php echo number_format($noteMoy, 2);?>
    <?php for ($i = 1; $i <= 5; $i++) { ?>
        <a class="aimer" href="./?action=noter&note=<?= $i ?>&idR=<?= $unResto->getIdR(); ?>" >
            <?php if ($i <= $noteMoy) { ?>
                <img class="note" src="images/like.png" alt="">
            <?php } else {
                ?>
                <img class="note" src="images/neutre.png" alt="line neutre">
            <?php } ?>
        </a>
    <?php } ?>
</span>
```

Ticket 8 : contrôle du format de saisie d code postal

Il nous est demander d'ajouter un contrôle lors du saisie du code postal pour que celui-ci comprenne un format précis de 5 chiffres, que ce soit pour une recherche simple ou multicritère.

SOLUTION :

Modification du champ d'insertion du code postale pour les recherches :

Initialement :

<input type="text" name="cpR" placeholder="Code postal" value="<?= $cpR ?>"/>

Apres modification :

<input type="text" name="cpR" placeholder="Code postal" value="<?= $cpR ?>" pattern="\d{5}" title="Le code postal doit contenir 5 chiffres" required />

Le champ pattern="\d{5}" oblige lutilisateur a rentrer 5 chiffres et rien d'autre

Ticket 9 :

Au lieu d'afficher une erreur SQL l'application doit reconnaitre que l'utilisateur veux inscrire un mail déjà utiliser.

SOLUTION :

Modification de la fonction pour insérer un utilisateur pour reconnaitre l'erreur de duplication d'un champ.

```
    public static function insert(Utilisateur $unUser): bool {
        $ok = false;
        try {
            $requete = "INSERT INTO utilisateur (mailU, pseudoU) VALUES (:mailU,:pseudoU)";
            $stmt = Bdd::getConnexion()->prepare($requete);
//            $mdpUCrypt = crypt($unUser->getMdpU(), "sel");
            $stmt->bindValue(':mailU', $unUser->getMailU(), PDO::PARAM_STR);
//            $stmt->bindValue(':mdpU', $mdpUCrypt, PDO::PARAM_STR);
            $stmt->bindValue(':pseudoU', $unUser->getPseudoU(), PDO::PARAM_STR);
            $ok = $stmt->execute();
        } catch (PDOException $e) {
            // 23000 = SQLSTATE pour "Integrity constraint violation"
            if ($e->getCode() === "23000") {
                throw new Exception(
                    "se mail a déja été utiliser !"
                );
            } else {
            throw new Exception("Erreur dans la méthode " . get_called_class() . "::insert : <br/>" . $e->getMessage());
            }
        }
        return $ok;
    }

    /**
```

Ticket 10 :

Contexte :

Il a été signalé qu'une attaque par injection SQL est possible sur la page de connexion.

Scénario : l'utilisateur saisit la chaîne de caractères suivants dans le champ de saisie de l'email : zzz' OR 1 = 1 ; DELETE FROM photo WHERE '1' = '1 et une valeur quelconque dans le mot de passe. L'application refuse l'authentification en affichant le message d'erreur.

Mais, ensuite, on peut constater que l'attaque a réussi, car **les photos des restaurants ne sont plus affichées** sur la page d'accueil (ni ailleurs) : les données de la table photo ont été supprimées !

On souhaite donc rendre impossibles les attaques par injection SQL sur ce formulaire

Repérage du problème :

Le problème provient du code UtilisateurDAO qui se trouve parmi les modèles DAO. En effet, dans ce code on définit la fonction "getOneByMail" qui permet de vérifier que le mail de l'utilisateur entré pour se connecter est bien le même que dans la BDD. Or cette requête est écrite d'une façon qui ne permet pas de bloquer les injections SQL.

```
try {
    $requete = "SELECT * FROM utilisateur WHERE mailU = '".$mailU."'";
    $stmt = Bdd::getConnexion()->query($requete);
```

Solution :

Pour régler ce problème, on modifie la requête afin d'en faire une requête préparée évitant ainsi la possibilité d'une injection SQL :

```
try {
    $requete = "SELECT * FROM utilisateur WHERE mailU = ?";
     $stmt = mysqli_prepare($requete);
     $mysqli_stmt_bind_param($stmt, 's', $mailU);
     $mysqli_stmt_execute($stmt);
     $result = mysqli_stmt_get_result($stmt);
     $row = mysqli_fetch_array($result);
     mysqli_stmt_close($stmt);
```

La fonction multi recherche est implémenter mais ne fonctionne pas nous devons trouver l'origine du problème

<u>SOLUTION :</u>

Modification de la function getAllMultiCriteres:

```
    */
    public static function getAllMultiCriteres(string $extraitNomR, string $voieAdrR, string $cpR, string $villeR): array {
        $lesObjets = array();
        try {
            $requete = "SELECT * FROM resto WHERE voieAdrR LIKE :voieAdrR AND cpR LIKE :cpR AND villeR LIKE :villeR AND nomR LIKE :no
            $stmt = Bdd::getConnexion()->prepare($requete);
            $motif = "%" . $extraitNomR . "%";
            $motifVoieAdrR = "%" . $voieAdrR . "%";
            $motifCpR = "%" . $cpR . "%";
            $motifVilleR = "%" . $villeR . "%";
            $stmt->bindParam(':nomR', $motif, PDO::PARAM_STR);
            $stmt->bindParam(':voieAdrR', $motifVoieAdrR, PDO::PARAM_STR);
            $stmt->bindParam(':cpR', $motifCpR, PDO::PARAM_STR);
            $stmt->bindParam(':villeR', $motifVilleR, PDO::PARAM_STR);
            $ok = $stmt->execute();
                // attention, $ok = true pour un select ne retournant aucune ligne
                if ($ok) {
                    // Pour chaque enregistrement
                    while ($enreg = $stmt->fetch(PDO::FETCH_ASSOC)) {
                        //Instancier un nouveau restaurant et l'ajouter à la liste
                        $lesObjets[] = self::enregistrementVersObjet($enreg);
                    }
                }

        } catch (PDOException $e) {
            throw new Exception("Erreur dans la méthode " . get_called_class() . "::getAllMultiCriteres : <br/>" . $e->getMessage());
        }
        return $lesObjets;
    }
```

sults Git Repository Browser 182:47/1:28 INS Unix (LF) PHP 7.4

Et donc de sa case qui l'appelle :

```
    case 'multi':
        // recherche multi-critere
        $listeRestos = RestoDAO::getAllMultiCriteres($nomR,$voieAdrR, $cpR, $villeR);
        break;

}
```